universität wien

MULTIOMICS DATA SCIENCE - STATISTICAL TESTING

EXCURSION - READIN & CLEANING OF TABLES

- Often tables contain (for a certain task) unnecessary columns or missing values
- Therefore, in a preprocessing step the table needs to be cleaned for follow-up analyses

EXCURSION - READIN & CLEANING OF TABLES

- Deleting columns:

```
# Let df be a priorly readin dataframe

# keep only certain columns
df_include = df.loc[:, [‘Column1‘, ‘Column2‘]]

# exclude certain columns
df_exclude = df.drop([‘Column1‘, ‘Column2‘], axis = 1)
```

EXCURSION - READIN & CLEANING OF TABLES

- Deleting **rows** which contain at least one NaN value:

```
# Let df be a priorly readin dataframe
df = df.dropna()
```

EXCURSION - READIN & CLEANING OF TABLES

- Deleting **rows** which contain at least one NaN value:

```
# Let df be a priorly readin dataframe
df = df.dropna(how = ‘any‘)  # same command with explicit instruction
```

EXCURSION - READIN & CLEANING OF TABLES

- Deleting **rows** which contain only NaN values:

```
# Let df be a priorly readin dataframe
df = df.dropna(how = ‘all‘)
```

EXCURSION - READIN & CLEANING OF TABLES

- Deleting **rows** in a certain subset of columns which contain at least one NaN value:

```
# Let df be a priorly readin dataframe
df = df.dropna(subset = ["Column1", "Column3"], how = ‘any‘)
```

EXCURSION - READIN & CLEANING OF TABLES

- Deleting **columns** in a certain subset of rows which contain at least one NaN value:

```
# Let df be a priorly readin dataframe
df = df.dropna(subset = ["Row1", "Row5"], how = ‘any‘, axis = 1)
```

EXCURSION - READIN & CLEANING OF TABLES

- Table transposing: columns will be rows, rows will be columns; it is like mirroring at the diagonally:

```
# Let df be a priorly readin dataframe
df = df.transpose()
```

BASICS

- Assumptions and hypotheses are being backuped with experiments in science
- However, empirical experiments can be biased or incorrect during acquisition
- To ensure significance, experiments are repeated
- Statistics offers (publically accepted) methods, to verify the results of these experiments (observations)

STATISTICAL TESTING

- In easy words: we are statistical detectives
- In a statistical test, we always assume the negative (boring) state, the so-called **null hypothesis**
- It is on us to show that the null hypothesis cannot apply

STATISTICAL TESTING

- In easy words: we are statistical detectives
- In a statistical test, we always assume the negative (boring) state, the so-called **null hypothesis**
- It is on us to show that the null hypothesis cannot apply
- Therefore, we use statistical tests, which compute p-values
- A p-value is the probability to obtain our observations under the condition that the null hypothesis is true

STATISTICAL TESTING

- We are given two samples $X = (X)_i$ and $Y = (Y)_j$
- We want to know if both samples are drawn from the same population

universität wien

STATISTICAL TESTING

- We are given two samples $X = (X)_i$ and $Y = (Y)_j$
- We want to know if both samples are drawn from the same population
- The t test is a statistical test to answer this questions
- The null hypothesis for the t test is that the mean values of both samples are identical, hence $\mu_x = \mu_y$
- Here: p-value = $P(X, Y \mid H_0 \text{ is true})$

universität wien

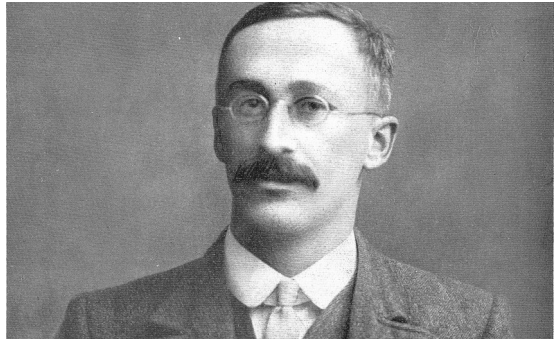
STATISTICAL TESTING

- The classical t test (Students test) assumes normally distributed and homoscedastical (same variance) data

- The Welch t test assumes normally distributed and heteroscedastical (different variance) data

STATISTICAL TESTING

- The classical t test (Students test) assumes normally distributed and homoscedastical (same variance) data
- The Welch t test assumes normally distributed and heteroscedastical (different variance) data
- The t test was developed by William Sealy Gosset who worked in the Guiness brewary on a method for controlling the quality of the stouts ;-) 🍺



Source: *http://unbiasedresearch.blogspot.com/2016/04/william-sealy-gosset-aka-student.html*

PAIRED VS UNPAIRED TESTING

- There are two t test versions, the paired and unpaired t test
- For a paired t test the observations need to be dependent on each other, e.g., measuring the heart beat / blood pressure before and after taking a medicine
- For a unpaired t test the observations need to be independent from each other, e.g., measuring the blood pressure of 5 healthy people and 5 patients

ONE-SIDED VS TWO-SIDED TESTING

- The t test distinguishes between three scenarios:
  - The first sample is **smaller** than the second sample (one-sided)
  - The first sample is **bigger** than the second sample (one-sided)
  - Both sample are **unequal** (two-sided)

STATISTICAL TESTING WITH PYTHON

- Unpaired Students t test with the `ttest_ind` function in SciPy:

```
from scipy import stats
import numpy as np

X = np.random.normal(20, 5, 15)          # Random numbers, please use
Y = np.random.normal(30, 7, 23)          # your true values here

tStat, pValue = stats.ttest_ind(X, Y)    # Students t test
print(pValue)                            # 0.000141480522
```

STATISTICAL TESTING WITH PYTHON

- Unpaired Students t test with the `ttest_ind` function in SciPy:

```
from scipy import stats
import numpy as np

X = np.random.normal(20, 5, 15)          # Random numbers, please use
Y = np.random.normal(30, 7, 23)          # your true values here

tStat, pValue = stats.ttest_ind(X, Y, alternative = 'less')
print(pValue)                            # 0.000141480522
```

- One-sided t tests are determined with the parameter `alternative` with the following values: {“`less`“, “`greater`“, “`two-sided`“ (default)}

STATISTICAL TESTING WITH PYTHON

- Unpaired Welch t test with the `ttest_ind` function in SciPy:

```
from scipy import stats
import numpy as np

X = np.random.normal(20, 5, 15)            # Random numbers, please use
Y = np.random.normal(30, 7, 23)            # your true values here

tStat, pValue = stats.ttest_ind(X, Y, equal_var = False)   # Welch T-Test
print(pValue)                              # 4.79291779e-05
```

STATISTICAL TESTING WITH PYTHON

- Paired Students t test with the `ttest_rel` function in SciPy:

```
from scipy import stats
import numpy as np

X = np.random.normal(20, 5, 15)          # Random numbers, please use
Y = np.random.normal(30, 7, 23)          # your true values here

tStat, pValue = stats.ttest_rel(X, Y)
print(pValue)                            # 4.79291779e-05
```

- There is no paired Welch t test

- One-sided t tests are determined with the parameter `alternative` with the following values: {“`less`“, “`greater`“, “`two-sided`“ (default)}

SIGNIFICANCE LEVEL

- To formulate a statement, we determine that a statistical test fails with an error probability of 5 % (here $\alpha = 0.05$)

- If the p-value is below α, we discard the null hypothesis

SIGNIFICANCE LEVEL

- To formulate a statement, we determine that a statistical test fails with an error probability of 5 % (here $\alpha = 0.05$)

- If the p-value is below α, we discard the null hypothesis

- The following significance definitions are excepted in the community:
  - p-value ≤ 5 %: significant (standard)

  - p-value ≤ 1 %: very significant

  - p-value ≤ 0.1 %: highly significant

ANOVA

- If we have more than two samples, the t test cannot be applied any more
- Therefore, the **AN**alysis **O**f **VA**riance was developd
- Requirement: alle samples are homoscedastically normally distributed
- The null hypothesis is that all sample means are equal, let: $\mu_1 = \mu_2 = \ldots = \mu_n$
- The ANOVA only recognizes, if at least one sample is different, not which one!

ANOVA

- In a one-way ANOVA the are several samples with only one parameter (e.g., body weight of 5 different cohorts)

- In a two-way ANOVA there are several samples with two different parameters (e.g., body weight and age of 5 different cohorts)

- In a three-way ANOVA ...

ANOVA WITH PYTHON

- ANOVA in SciPy:

```
from scipy import stats
import numpy as np

X = np.random.normal(20, 5, 15)          # Random numbers, please use
Y = np.random.normal(30, 7, 23)          # your true values here
Z = np.random.normal(30, 7, 23)          # and here too

tStat, pValue = stats.f_oneway(X, Y, Z)
print(pValue)                            # 1.3622377e-05
```

- To determine which sample follows a different population, a posthoc Test such as Tukey honest significance difference (HSD) can be used

MULTIPLE TESTING

- Often we don‘t do one t test per experiment in (analytica) chemistry, but several
- The `ttest_ind` function can apply several tests at once

MULTIPLE TESTING

- Often we don‘t do one t test per experiment in (analytica) chemistry, but several
- The `ttest_ind` function can apply several tests at once
- Therefore, two sample matrices must be present
- The samples must be provided column wise in a matrix
- Both matrices must therefore have the same number of columns but can have a different number of rows

MULTIPLE TESTING

But I am making only one big testing, exactly as I am doing only one global experiment?!?

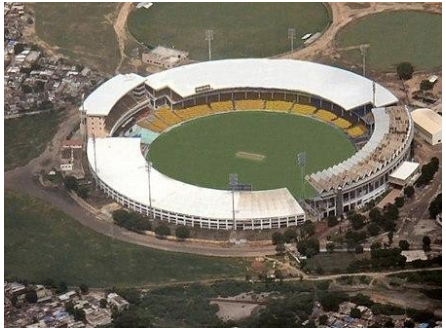
MULTIPLE TESTING

But I am making only one big testing, exactly as I am doing only one global experiment?!?

May be, but you obtain *n* p-values, hence multiple testing!

EXPERIMENT FOR DEMONSTRATION

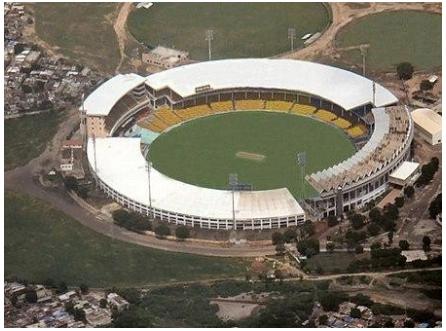
- The Narendra Modi stadion in Ahmedabad / India is the biggest stadion in the world and has space for 132.000 guests
- In the completely filled stadion, we ask all people to flip a coin exactly 15x
- All people should raise their hand, who threw 15x head



Quelle: *https://de.wikipedia.org/wiki/Narendra_Modi_Stadium*

EXPERIMENT FOR DEMONSTRATION

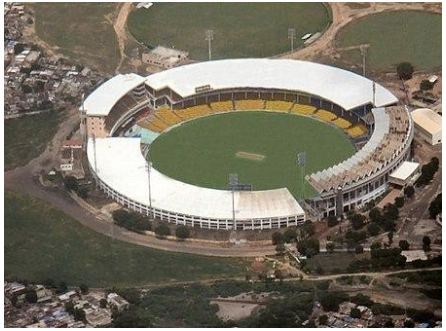
- The Narendra Modi stadion in Ahmedabad / India is the biggest stadion in the world and has space for 132.000 guests
- In the completely filled stadion, we ask all people to flip a coin exactly 15x
- All people should raise their hand, who threw 15x head
- Indeed, three people managed this little wonder
- Do these three people have the “**golden hand**“?



Quelle: *https://de.wikipedia.org/wiki/Narendra_Modi_Stadium*

EXPERIMENT FOR DEMONSTRATION

- The Narendra Modi stadion in Ahmedabad / India is the biggest stadion in the world and has space for 132.000 guests
- In the completely filled stadion, we ask all people to flip a coin exactly 15x
- All people should raise their hand, who threw 15x head
- Indeed, three people managed this little wonder
- Do these three people have the "**golden hand**"?
- Expected value: $132000 \cdot 1 / 2^{15} \approx 4$, hence not really!



Quelle: *https://de.wikipedia.org/wiki/Narendra_Modi_Stadium*

PROBLEM WITH MULTIPLE TESTING

- With $\alpha = 0.05$ we accept a false result with 5 % during testing
- If we perform only one test, it‘s fine
- If we perform 1000 tests, the probability rises that at least one (or more) test will fail

PROBLEM WITH MULTIPLE TESTING

- With $\alpha = 0.05$ we accept a false result with 5 % during testing
- If we perform only one test, it‘s fine
- If we perform 1000 tests, the probability rises that at least one (or more) test will fail
- Prob. that no test fails performing 100 tests with $\alpha = 0.05$: $0.95^{100} = 0.00592$
- Prob. that no test fails performing 1000 Tests with $\alpha = 0.05$ is about $5 \cdot 10^{-23}$

MULTIPLE TESTING

- Example:

```
from scipy import stats
import numpy as np

sample1 = np.random.rand(7, 100)          # Random matrices, please us
sample2 = np.random.rand(9, 100)          # here your true data
tStats, pValues = stats.ttest_ind(sample1, sample2)

print(pValues)
# [0.61715259 0.32397673 .. 0.57412207 0.70030239]
```

- Caution: `ttest_ind` compares column wise, the number of columns needs to be equal in both matrices!

MULTIPLE TESTING

- Example:

```
from scipy import stats
import numpy as np

sample1 = np.random.rand(7, 100)          # Random matrices, please us
sample2 = np.random.rand(9, 100)          # here your true data
# Ignore NaN values
tStats, pValues = stats.ttest_ind(sample1, sample2, nan_policy = 'omit')

print(pValues)
# [0.61715259 0.32397673 .. 0.57412207 0.70030239]
```

- Caution: `ttest_ind` compares column wise, the number of columns needs to be equal in both matrices!

P-VALUE CORRECTION

- During a p-value correction, the p-values are adjusted with respect to the number of executed tests
- In a Bonferroni correction, all p-values are multiplied by the number of tests
- This correction is the strictest and the most naive correction

P-VALUE CORRECTION

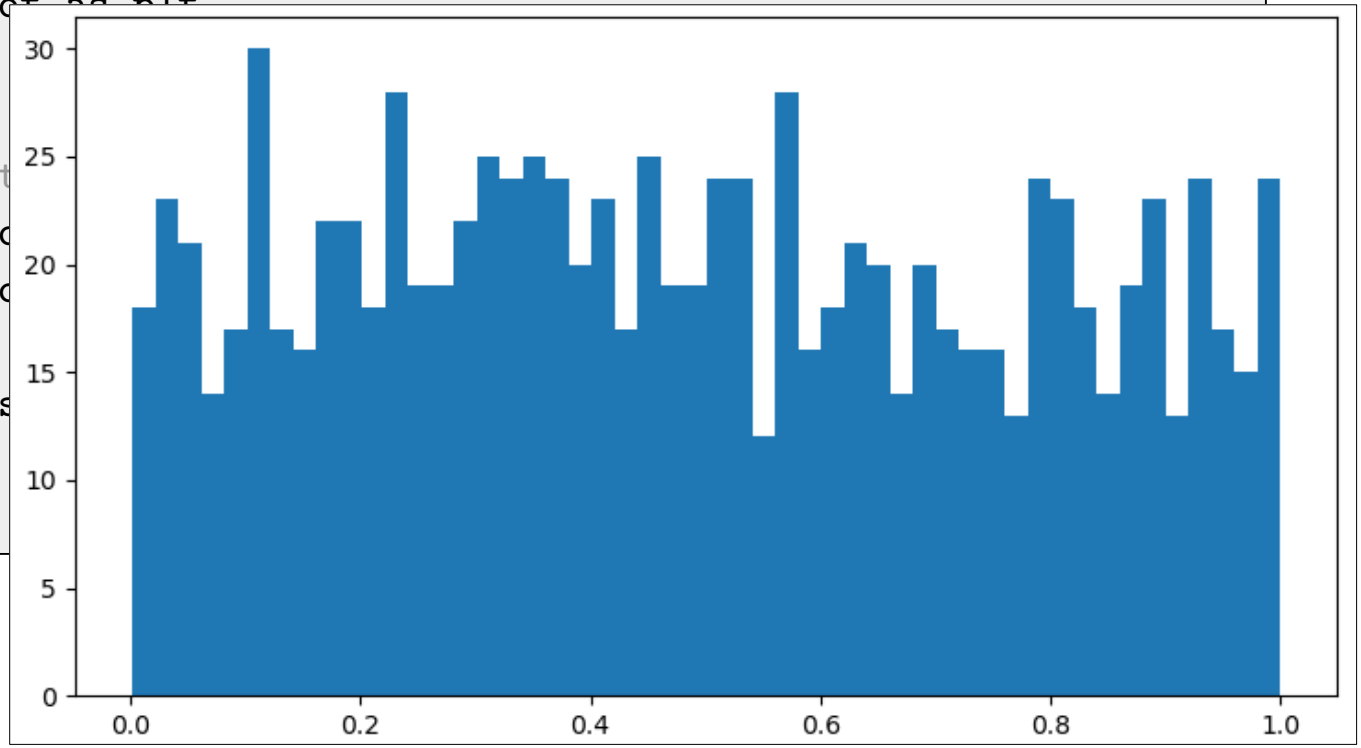
- During a p-value correction, the p-values are adjusted with respect to the number of executed tests
- In a Bonferroni correction, all p-values are multiplied by the number of tests
- This correction is the strictest and the most naive correction
- The mathematicians Benjamini and Hochberg have developed a more clever correction
- Let‘s have a look at typical p-value distributions

P-VALUE DISTRIBUTION - BACKGROUND NOISE

```
from scipy import stats
import numpy as np
from numpy.random import normal as norm
import matplotlib.pyplot as plt

# 1000 samples with 15 observations each
# from the same population in two samples
sample1 = np.matrix([norm(20, 5, 15) for i in range(1000)]).transpose()
sample2 = np.matrix([norm(20, 5, 15) for i in range(1000)]).transpose()

tStats, pValues = stats.ttest_ind(sample1, sample2)
plt.hist(pValues, bins = 50)
plt.show()
```

P-VALUE DISTRIBUTION - BACKGROUND NOISE

```
from scipy import stats
import numpy as np
from numpy.random import normal as norm
import matplotlib.pyplot as plt

# 1000 samples with 15
# from the same populat
sample1 = np.matrix([no
sample2 = np.matrix([no

tStats, pValues = stats
plt.hist(pValues, bins
plt.show()
```



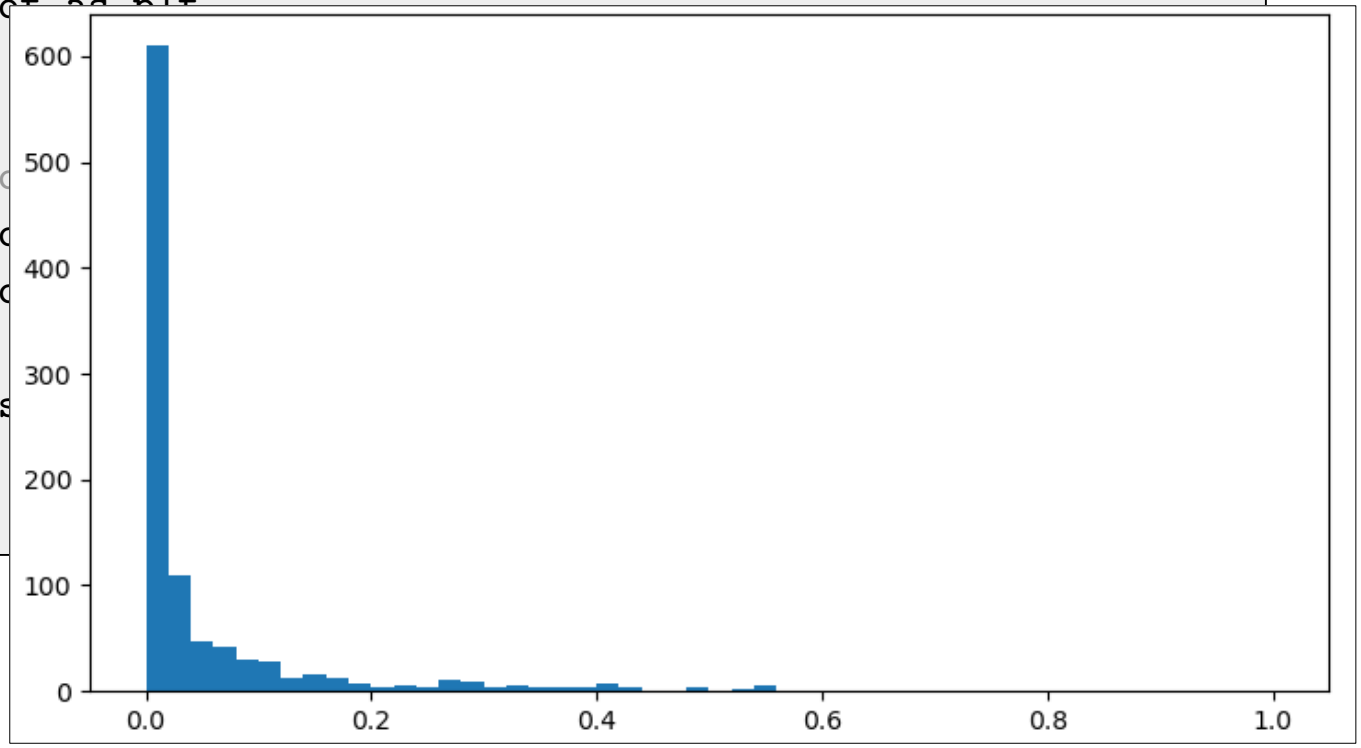
universität wien

P-VALUE DISTRIBUTION - SIGNAL

```
from scipy import stats
import numpy as np
from numpy.random import normal as norm
import matplotlib.pyplot as plt

# 1000 samples with 15 observations each
# from two different populations in two samples
sample1 = np.matrix([norm(20, 5, 15) for i in range(1000)]).transpose()
sample2 = np.matrix([norm(25, 5, 15) for i in range(1000)]).transpose()

tStats, pValues = stats.ttest_ind(sample1, sample2)
plt.hist(pValues, bins = 50)
plt.show()
```

P-VALUE DISTRIBUTION - SIGNAL

```
from scipy import stats
import numpy as np
from numpy.random import normal as norm
import matplotlib.pyplot as plt

# 1000 samples with 15
# from two different po
sample1 = np.matrix([no
sample2 = np.matrix([no

tStats, pValues = stats
plt.hist(pValues, bins
plt.show()
```



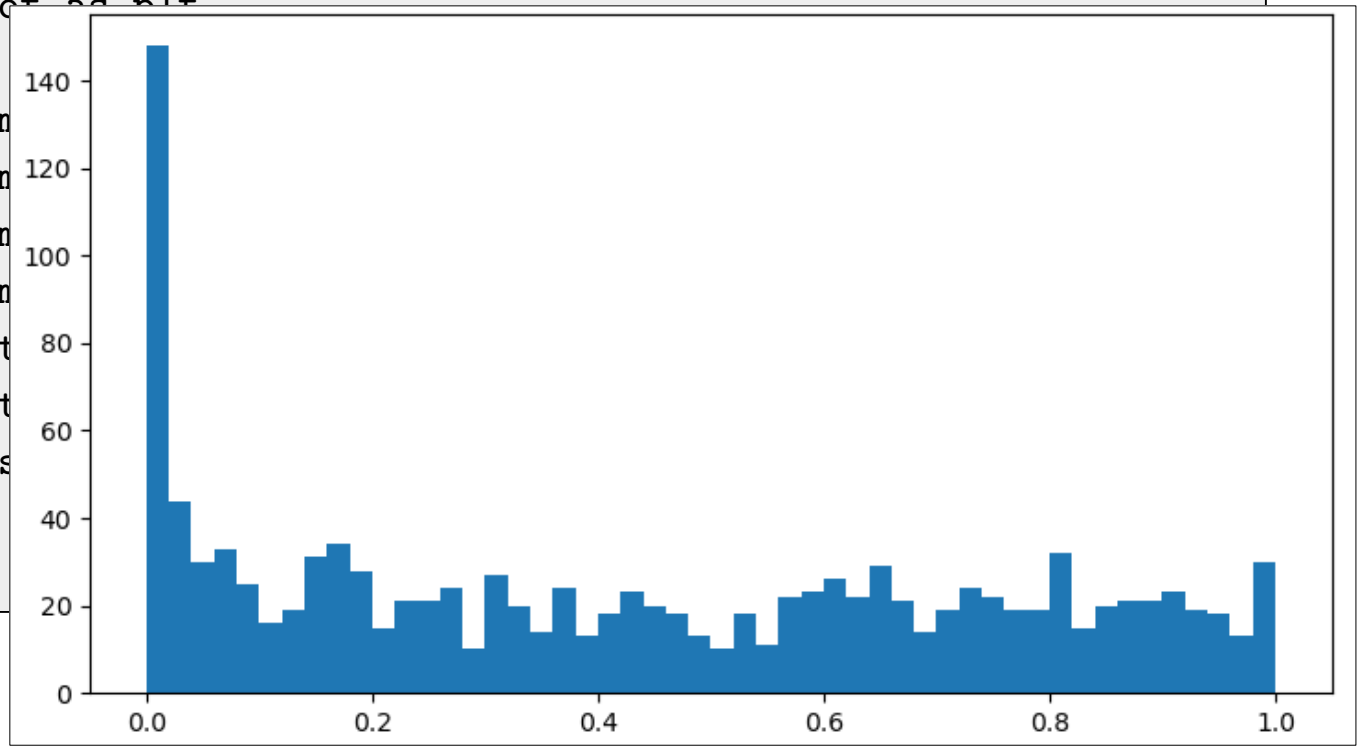
universität wien

P-VALUE DISTRIBUTION - SIGNAL & NOISE

```
from scipy import stats
import numpy as np
from numpy.random import normal as norm
import matplotlib.pyplot as plt

sig_1 = np.matrix([norm(20, 5, 15) for i in range(200)]).transpose() # Sig
noi_1 = np.matrix([norm(40, 5, 15) for i in range(1000)]).transpose() # Noi
sig_2 = np.matrix([norm(25, 5, 15) for i in range(200)]).transpose() # Sig
noi_2 = np.matrix([norm(40, 5, 15) for i in range(1000)]).transpose() # Noi
sample1 = np.concatenate((sig_1, noi_1), axis = 1)
sample2 = np.concatenate((sig_2, noi_2), axis = 1)
tStats, pValues = stats.ttest_ind(sample1, sample2)
plt.hist(pValues, bins = 50)
plt.show()
```

P-VALUE DISTRIBUTION - SIGNAL & NOISE

```
from scipy import stats
import numpy as np
from numpy.random import normal as norm
import matplotlib.pyplot as plt

sig_1 = np.matrix([norm
noi_1 = np.matrix([norm
sig_2 = np.matrix([norm
noi_2 = np.matrix([norm
sample1 = np.concatenat
sample2 = np.concatenat
tStats, pValues = stats
plt.hist(pValues, bins
plt.show()
```



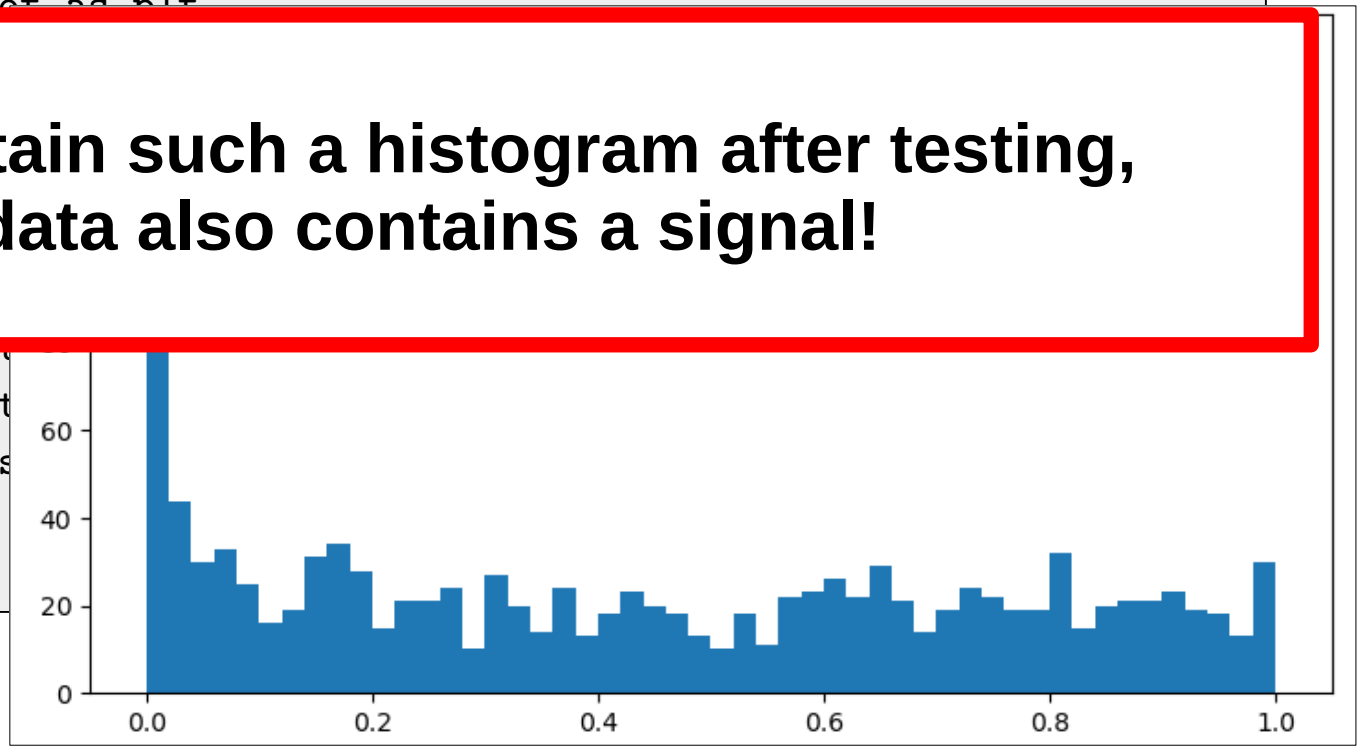
universität wien

P-VALUE DISTRIBUTION - SIGNAL & NOISE

```
from scipy import stats
import numpy as np
from numpy.random import normal as norm
import matplotlib.pyplot as plt
```

When you obtain such a histogram after testing, your data also contains a signal!

```
sample1 = np.concatenat
sample2 = np.concatenat
tStats, pValues = stats
plt.hist(pValues, bins
plt.show()
```



universität wien

MULTIPLE CORRECTION

- Example with Bonferroni correction:

```
from scipy import stats
import numpy as np
from statsmodels.stats.multitest import multipletests

sample1, sample2 = np.random.rand(7, 100), np.random.rand(9, 100)
tStats, pValues = stats.ttest_ind(sample1, sample2)

pValues_corr = multipletests(pValues, method = 'bonferroni')[1]
print(pValues, pValues_corr)
# [0.61715259 0.32397673 .. 0.05712207 0.70030239]
# [1.000 1.000 1.000 .. 1.000, 1.000, 1.000]
```

MULTIPLE CORRECTION

- Example with Benjamini-Hochberg correction:

```
from scipy import stats
import numpy as np
from statsmodels.stats.multitest import multipletests

sample1, sample2 = np.random.rand(7, 100), np.random.rand(9, 100)
tStats, pValues = stats.ttest_ind(sample1, sample2)

pValues_corr = multipletests(pValues, method = 'fdr_bh')[1]
print(pValues, pValues_corr)
# [0.61715259 0.32397673 .. 0.05712207 0.70030239]
# [0.96934303 0.9642187 .. 0.60993971 0.8968144]
```

PREPARATION FOR STATISTICAL TESTING

- For an empical experiment with additional statistical analysis, we need a good preparation
- For instance, when comparing two samples with two values in each sample, no significant statements can be made
- On the other hand, it is often not possible to repeat a measurement 1000 times per condition

PREPARATION FOR STATISTICAL TESTING

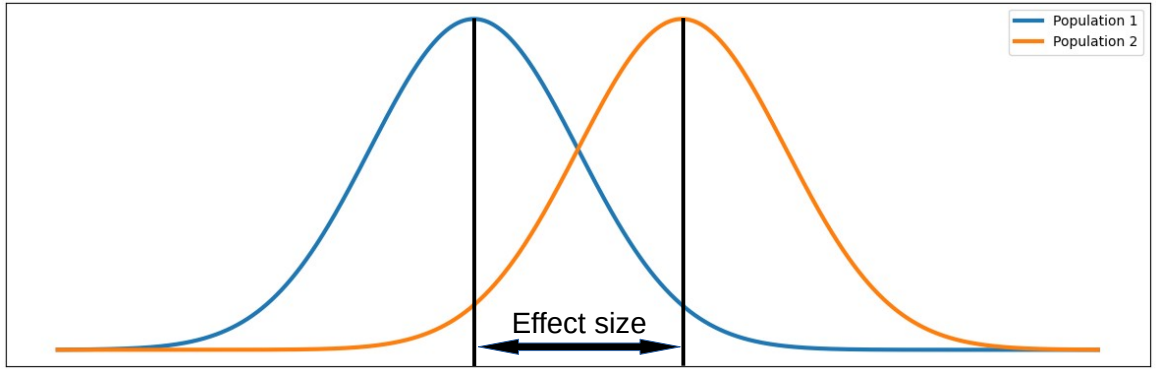
- For an empical experiment with additional statistical analysis, we need a good preparation
- For instance, when comparing two samples with two values in each sample, no significant statements can be made
- On the other hand, it is often not possible to repeat a measurement 1000 times per condition
- The power analysis offers a statistical method, to plan statistical testing

POWER ANALYSIS

- The power analysis combined four measuses of statistical testing:
  - Sample size: number of measurements
  - Effect size: next slide
  - Signifikance level: the α valie
  - Statistical power: probability to correcty reject the null hypothesis, let P(reject H_0 | H_1 is true)

EFFECT SIZE

- Measure for the strength of the phenomenon to be tested / investigated

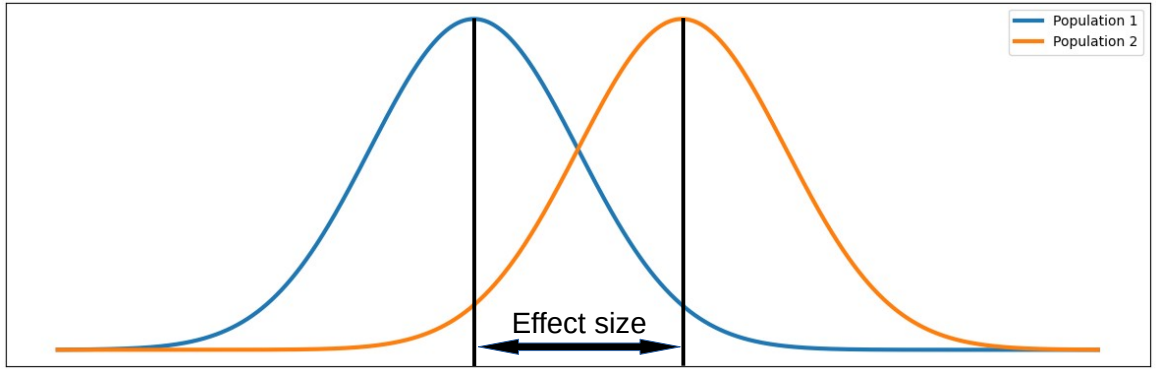


- Typical values:
  - low effect: 0.2
  - medium effect: 0.5
  - high effect: 0.8 (standard)

universität wien

EFFECT SIZE

- Measure for the strength of the phenomenon to be tested / investigated



- Typical values:
  - low effect: 0.2
  - medium effect: 0.5
  - high effect: 0.8 (standard)

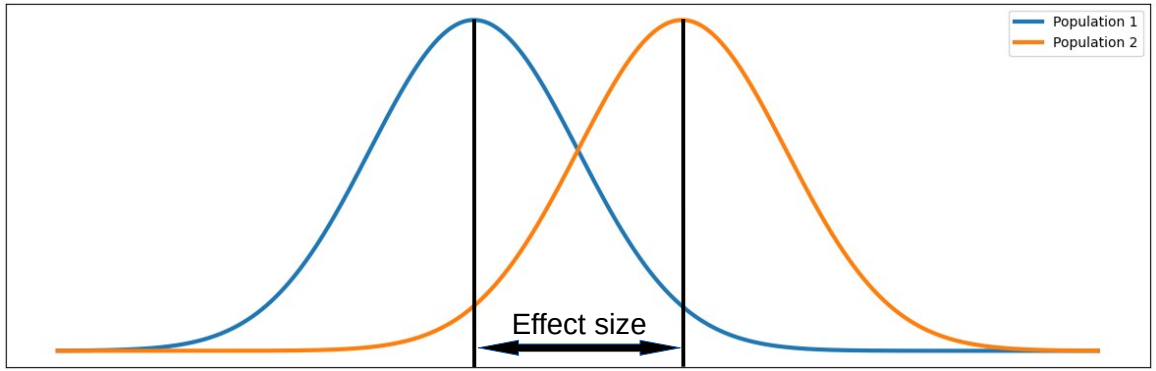
Cohens effect size:

$$D = \frac{\mu_2 - \mu_1}{\sigma}$$

universität wien

EFFECT SIZE

- Measure for the strength of the phenomenon to be tested / investigated



- Typical values:
  - low effect: 0.2
  - medium effect: 0.5
  - high effect: 0.8 (standard)

Cohens effect size:

$$D = \frac{\mu_2 - \mu_1}{\sigma}$$

Recall: same variance

universität wien

POWER ANALYSIS WITH PYTHON

- The function `solve_power` computes the fourth measure based on the remaining the measures:

```
from statsmodels.stats.power import TTestIndPower

d, power, alpha = 0.8, 0.9, 0.05
obj = TTestIndPower()
nobs1 = obj.solve_power(effect_size = d, alpha = alpha, power = power,
nobs1 = None)
print(nobs1)   # 33.82 measurements per samples
```

POWER ANALYSIS WITH PYTHON

- The function `solve_power` computes the fourth measure based on the remaining the measures:

```
from statsmodels.stats.power import TTestIndPower

d, power, alpha = 0.8, 0.9, 0.05
obj = TTestIndPower()
nobs1 = obj.solve_power(effect_size = d, alpha = alpha, power = power,
nobs1 = None, ratio = 1.5)  # ratio between samples
print(nobs1)   # 28.15 measurements for the first sample
```

POWER ANALYSIS WITH PYTHON

- The function `solve_power` computes the fourth measure based on the remaining the measures:

```
from statsmodels.stats.power import TTestIndPower

d, nobs1, alpha = 0.8, 10, 0.05
obj = TTestIndPower()
power = obj.solve_power(effect_size = d, alpha = alpha, power = None,
nobs1 = nobs1)
print(power)   # 0.39 stat. Power
```

POWER ANALYSIS WITH PYTHON

- For an ANOVA, the function `solve_power` computes the 5 measure based on the four remaining measures:

```
from statsmodels.stats.power import FTestAnovaPower

d, power, alpha, k_groups = 0.8, 0.95, 0.05, 3
obj = FTestAnovaPower()
nobs = obj.solve_power(effect_size = d, alpha = alpha, power = power,
nobs = None, k_groups = k_groups)
print(nobs)   # 27.4 measurements for all samples
```

POWER ANALYSIS WITH PYTHON

- For an ANOVA, the function `solve_power` computes the 5 measure based on the four remaining measures:

```
from statsmodels.stats.power import FTestAnovaPower

d, nobs, alpha, k_groups = 0.8, 35, 0.05, 3
obj = FTestAnovaPower()
power = obj.solve_power(effect_size = d, alpha = alpha, power = None,
nobs = nobs, k_groups = k_groups)
print(power)   # 0.986 stat. Power
```